Course Cheat Sheet

Develop Self-Improving AI Agents with RL

O'REILLY®

Contents

Section 1: Introduction to LLM Reinforcement Learning

Core idea: Reinforcement learning formalizes how an agent improves through interaction. For LLM agents, the optimized object is the full trajectory of reasoning and actions, not only the final answer.

1. Intuition: beyond zero sum games

Reinforcement learning is the formal version of a feedback loop you already know: an agent tries an action, observes what happened, then adjusts based on whether the outcome was good or bad. This is the bridge from rewards to reasoning.

2. The RL interaction loop (MDP framing)

A decision process can be framed as an MDP with:

- State s
- Action a
- Transition to next state s'
- Reward r

The loop:

- The agent observes the current state s
- The policy selects an action a
- The environment returns reward r and next state s'
- The policy is updated so future actions improve

MDP interaction loop

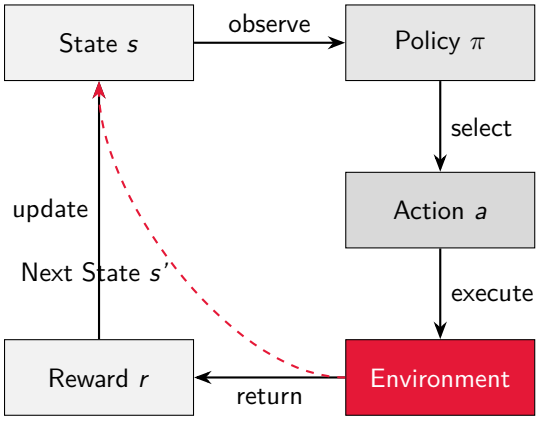

Figure 1: The MDP interaction loop showing how state, policy, action, environment, and reward form a continuous learning cycle.

Code example: MDP structure in Frozen Lake

```
# State: current board position
game = {
    "position": (0, 0),  # State s
    "grid": [["S", "F", "F", "F"], ...]
}

# Action: agent chooses direction
Action = Literal["up", "down", "left", "right"]

# Reward: based on outcome
if tile == "G":
    reward = 1.0  # Success
elif tile == "H":
    reward = -1.0  # Failure
reward -= 0.01 * steps  # Efficiency penalty
```

3. Policy as a probabilistic decision function

For a given state s, a policy $\pi(\cdot \mid s)$ outputs a probability distribution over actions.

- High probability actions represent preferred behaviors
- Low probability actions represent less favored behaviors

Learning shifts probability mass toward actions that lead to higher reward. This is behavior shaping through probability updates.

Policy probability distribution

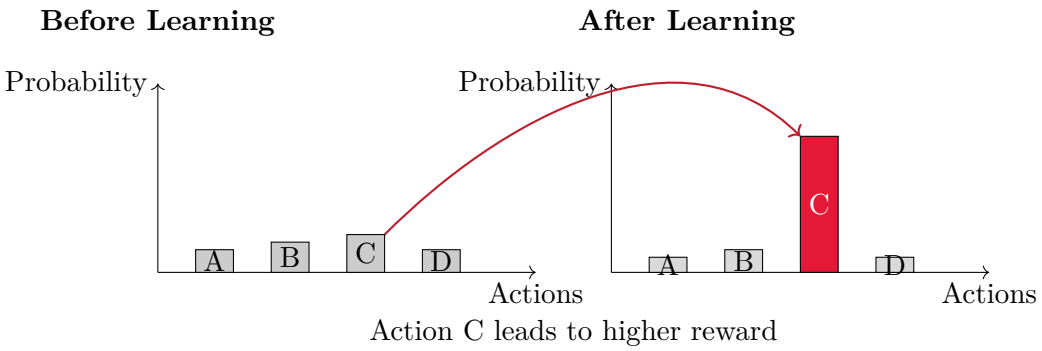

Figure 2: Policy learning shifts probability mass from uniform distribution toward actions that lead to higher rewards.

Code example: Policy as probability distribution

```python
def softmax(logits: Dict[str, float],
            temperature: float = 1.0) -> Dict[str, float]:
    """Convert logits to probabilities using softmax."""
    m = max(logits.values())
    exps = {k: math.exp((v - m) / temperature)
            for k, v in logits.items()}
    s = sum(exps.values())
    return {k: exps[k] / s for k in exps}

# Policy outputs probability distribution
logits = {"plan": 0.2, "direct_answer": 0.5,
          "verify": 0.3, "use_tool": 0.1}
probs = softmax(logits)  # (·|s)
# High probability actions = preferred behaviors
```

4. Where learning happens for LLM agents

The key shift for LLM agents is that learning is about process. A rollout is a complete story of reasoning, not a single label.

Concept	Meaning
Reward	Signal indicating whether an outcome was good or bad
Trajectory	Full sequence of reasoning steps, actions, and tool calls that lead to an outcome
Rollout	One complete execution of the policy for a given prompt, producing a full trajectory
Policy	Internal decision rule mapping context to the next action
Reward or judge model	Evaluates trajectories and assigns reward based on quality or correctness
Relative evaluation	Compares multiple rollouts to decide which behavior was better

Code example: Trajectory structure

```python
@dataclass
class Rollout:
    """A single rollout: prompt, action, transcript, reward."""
    prompt_id: int
    prompt: str
    action: str
    transcript: str  # Full reasoning sequence
    reward: float
    step: int

# Trajectory captures the full reasoning process
trajectory = art.Trajectory(
```

```
    messages_and_choices=[
        {"role": "system", "content": "..."},
        {"role": "user", "content": board_state},
        {"role": "assistant", "content": "<move>up</move>"},
        # ... more reasoning steps
    ],
    reward=1.0,  # Evaluated after completion
    metadata={"game_id": "...", "outcome": "success"}
)
```

5. Acting versus learning (two phase view)

Acting phase:

- Policy produces a trajectory
- A completed trajectory becomes a rollout

Learning phase:

- A reward or judge model evaluates the rollout and assigns reward
- Relative evaluation compares rollouts to identify better behavior
- The policy updates from that feedback, closing the loop

Code example: Acting and learning phases

```
# ACTING PHASE: Generate rollouts
for step in range(num_steps):
    # Generate multiple rollouts in parallel
    train_groups = await art.gather_trajectory_groups(
        (rollout(model, scenario) for _ in range(18))
    )

    # LEARNING PHASE: Evaluate and update
    # Rewards are computed during rollout
    # Policy updates based on trajectory rewards
    await model.train(
        train_groups,
        config=art.TrainConfig(learning_rate=1e-5)
    )
```

6. From individual steps to a complete rollout

An episode begins when the policy chooses the first action. Each action execution adds a step to the trajectory. When the task is complete:

- The final trajectory is captured as a rollout
- The rollout becomes the unit that is evaluated and later used for learning

Code example: Rollout function generating a trajectory

```python
@art.retry
async def rollout(model: art.Model,
                  scenario: ScenarioFrozenLake) -> art.Trajectory:
    game = generate_game()  # Initialize state
    trajectory = art.Trajectory(messages_and_choices=[...])

    while not check_game_finished(game):
        # Agent observes state
        board_state = render_board(game)
        trajectory.messages_and_choices.append(
            {"role": "user", "content": board_state}
        )

        # Policy selects action
        response = await model.generate(messages)
        action = parse_move(response)

        # Environment transitions
        apply_agent_move(game, action)
        trajectory.messages_and_choices.append(response)

    # Reward assigned at completion
    trajectory.reward = compute_reward(game)
    return trajectory  # Complete rollout
```

7. The missing piece: why rollouts matter

- Supervised learning teaches outputs
- Reinforcement learning teaches process
- Rollouts are entire stories of reasoning
- Comparing multiple rollouts gives the model a sense of better versus worse

Code example: Reward computation from complete trajectory

```python
def score_rollout(transcript: str, correct_answer: str,
                  action: str) -> float:
    """Compute reward for a complete rollout."""
    # Correctness signal (most important)
    correct = 1.0 if correct_answer in transcript else 0.0

    # Efficiency penalty (discourage verbosity)
    length_penalty = 0.0009 * len(transcript)

    # Shaping bonuses (encourage helpful behaviors)
    tool_bonus = 0.06 if "Tool:" in transcript else 0.0
    verify_bonus = 0.04 if "Verification" in transcript else 0.0
```

```
    # Refusal penalty
    refuse_penalty = 0.7 if action == "refuse" else 0.0

    reward = (correct + tool_bonus + verify_bonus
              - length_penalty - refuse_penalty)
    return max(-1.0, min(1.0, reward))
```

8. Key takeaways

- Reinforcement learning lets agents learn from experience through interaction
- Trajectories and rollouts matter because they capture the reasoning and action process, not only the final answer
- Policies are probabilistic and learning shifts action probabilities toward behaviors that produce better outcomes
- Rewards connect actions to consequences by scoring trajectory quality and guiding improvement
- Supervised data alone can't teach open ended reasoning, tool use, or continual improvement

Section 2: Reward Modeling and Relative Scoring

Core idea: Reward functions shape agent behavior. For open ended agent tasks, relative evaluation is more stable and generalizable than absolute scoring.

1. How reward functions shape behavior

A reward function determines which behaviors the agent reinforces. Over repeated learning cycles:

- Behaviors associated with higher reward become more likely
- The policy gradually shifts toward those behaviors

Rewards close the loop between action and consequence. Without reward, the agent has no signal for progress.

Code example: Simple reward function

```
def judge_countdown_expression(
    target: int,
    allowed_nums: List[int],
    expr: str,
) -> Tuple[float, Optional[float], Optional[str]]:
    """Returns (reward, value, reason)."""
    # Check if expression evaluates to target
    val = safe_eval_expr(expr)
    if abs(val - target) < 1e-9:
        return 1.0, val, None  # Success
    return 0.0, val, "wrong value"  # Failure
```

2. Reward hacking

Reward hacking occurs when an agent exploits weaknesses in the reward function instead of learning the intended behavior.

Common causes:

- Overly simplistic or brittle numeric metrics
- Proxy objectives that are easier to optimize than the real goal
- Absolute scores that can be gamed without improving true task quality

Reward hacking is not a bug in the agent. It is a mis specification of the learning signal.

Code example: Brittle reward function prone to hacking

```
def static_reward_function(response_content, task_type="poem"):
    """A naive static reward - easily gamed!"""
    score = 0.0

    # Length-based scoring (bad heuristic!)
    if len(response_content) > 200:
        score += 0.3
    elif len(response_content) > 100:
        score += 0.2

    # Keyword matching (bad heuristic!)
    if "cat" in response_content.lower():
        score += 0.3
    if "star" in response_content.lower():
        score += 0.2

    # Line count (bad heuristic!)
    if response_content.count('\n') >= 4:
        score += 0.2

    return min(score, 1.0)
# Model can game this by adding filler text and keywords!
```

3. Limits of absolute scoring

Absolute reward values are hard to design for language based agents because:

- Many tasks have no single correct answer
- Qualities like reasoning quality, coherence, or strategy are subjective
- LLM judges are poorly calibrated when scoring outputs in isolation

As a result:

- Scores become noisy and inconsistent
- Learning becomes unstable

Code example: Absolute scoring inconsistency

```
# Same response gets different absolute scores
# depending on judge model calibration

response = "Cats sit and look at stars at night"

# Judge A (strict): score = 0.3
# Judge B (lenient): score = 0.8
# Judge C (miscalibrated): score = 0.1

# Problem: Which score is "correct"?
# Solution: Use relative ranking within groups
```

4. Relative scoring: the key shift

Relative scoring replaces absolute metrics with comparison.

Instead of asking:

> "How good is this output?"

The system asks:

> "Which of these outputs is better?"

Learning only requires consistency within a group, not globally calibrated scores.

Absolute vs Relative scoring

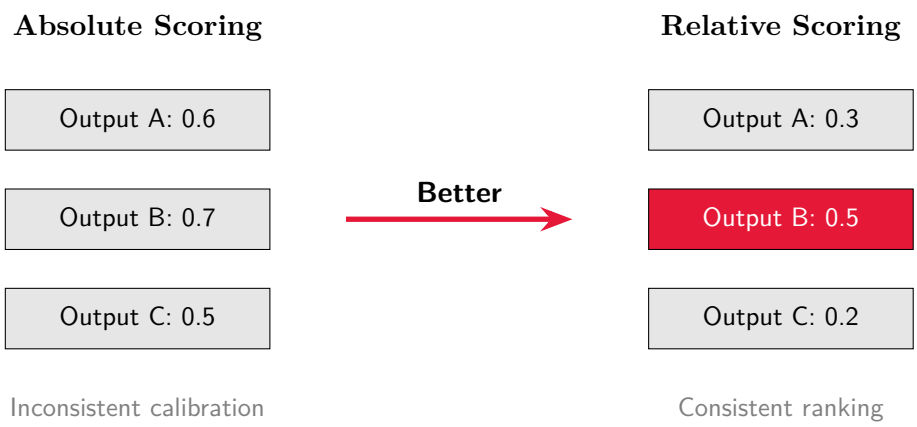

Figure 3: Relative scoring provides consistent ranking within groups, avoiding calibration issues of absolute scores.

Code example: Relative scoring with RULER

```
# Instead of asking "How good is this?" (absolute)
# We ask "Which is better?" (relative), group trajectories by same prompt
group = art.TrajectoryGroup([
    good_trajectory,
    mediocre_trajectory,
    off_topic_trajectory
])

# RULER ranks them relative to each other
judged_group = await ruler_score_group(
    group,
    "openai/gpt-4.1",
    debug=True
)

# Scores are relative: best in group gets highest score
# regardless of absolute quality level
```

5. Why relative ranking is more stable

Relative ranking improves learning because:

- The judge compares rollouts side by side
- Differences are easier to detect than absolute quality
- Calibration errors cancel out within a group
- The learning signal reflects preferences, not fixed labels

This is especially effective for open ended tasks with multiple valid solutions.

Code example: Group-based relative ranking

```
# RULER evaluates trajectories together
async def compare_trajectories():
    # Three different quality responses
    excellent = art.Trajectory(...)  # Score: 0.9
    good = art.Trajectory(...)       # Score: 0.6
    poor = art.Trajectory(...)       # Score: 0.1

    group = art.TrajectoryGroup([excellent, good, poor])
    judged = await ruler_score_group(group, "openai/o3")

    # Relative scores show clear ranking
    sorted_trajs = sorted(
        judged.trajectories,
        key=lambda t: t.reward,
        reverse=True
    )
```

6. Examples of open ended agent tasks

Relative scoring is critical for tasks such as:

- Tool use decisions (when and how to call tools)
- Strategy generation instead of answer generation
- Multi step planning and troubleshooting
- Customer support decisions with ambiguous inputs
- Multi turn conversations with evolving context

These tasks can't be reliably evaluated with binary correctness.

Code example: Open-ended task evaluation

```
# Example: Countdown math puzzle solving
# Multiple valid expressions can reach the target

scenario = Scenario(
    target=100,
    nums=[25, 4, 3, 2],
    question="Find expression that equals 100"
)

# Valid solutions:
# "(25 * 4)" = 100
# "(25 * 3) + 25" = 100
# "(4 * 25)" = 100

# RULER evaluates quality, not just correctness:
# - Uses fewer operations? (better)
# - Avoids redundant parentheses? (better)
# - Cleaner expression? (better)
```

7. What is RULER

RULER (Relative Universal LLM Elicited Rewards) is a general purpose reward function that:

- Requires no labeled data
- Requires no handcrafted reward functions
- Requires no human feedback

RULER evaluates agent behavior by comparing multiple trajectories produced for the same prefix.

Code example: RULER usage

```
# RULER requires no labels, no handcrafted functions

trajectories = [
    art.Trajectory(messages_and_choices=[...], reward=0.0),
    art.Trajectory(messages_and_choices=[...], reward=0.0),
    art.Trajectory(messages_and_choices=[...], reward=0.0),
]

group = art.TrajectoryGroup(trajectories)

# RULER automatically ranks them
judged_group = await ruler_score_group(
    group,
    "openai/gpt-4.1",  # Judge model
    debug=True
)
# Each trajectory now has a relative reward score
```

8. How RULER works at a high level

- The agent generates multiple rollouts for the same task
- A judge model evaluates the trajectories together
- The trajectories are ranked relative to each other
- Rankings are converted into reward signals
- These rewards are used to update the policy

A single trajectory shows what happened. A group of trajectories shows what could have happened.

Code example: RULER workflow

```
# Step 1: Generate multiple rollouts for same task
groups = []
for scenario in scenarios:
    group = art.TrajectoryGroup([
        rollout(model, scenario)
        for _ in range(3)  # 3 rollouts per prompt
    ])
    groups.append(group)

# Step 2: Judge model evaluates trajectories together
judged_groups = []
for group in groups:
    judged = await ruler_score_group(
        group,
        "openai/gpt-4.1"
    )
```

```
    judged_groups.append(judged)

# Step 3: Rankings converted to reward signals
# Step 4: Rewards used to update policy
await model.train(judged_groups, ...)
```

RULER workflow

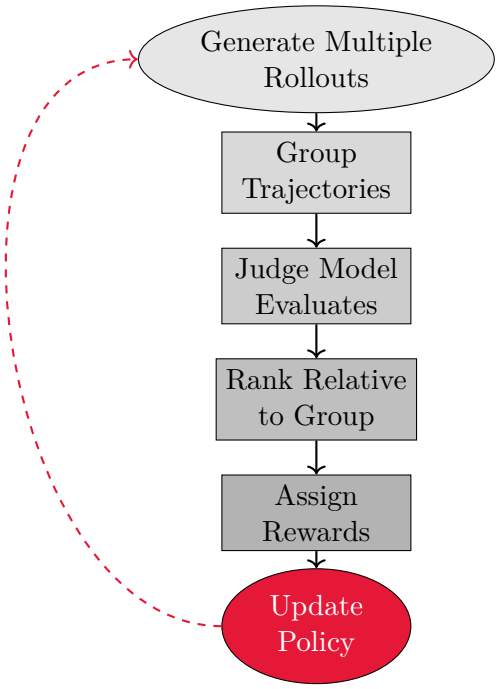

Figure 4: RULER workflow: generate rollouts, group trajectories, evaluate with judge model, rank relatively, assign rewards, and update policy.

9. Key properties of RULER

Property	Why it matters
Group judging	Easier detection of quality differences
Relative ranking	Removes dependence on absolute score calibration
Preference learning	Aligns with open ended task structure
No human labels	Enables scalable self improvement

Code example: Combining hard correctness with RULER

```
# Hybrid reward: gate * (alpha + beta * ruler_norm)
def combine_rewards(groups, alpha=0.7, beta=0.3):
    for g in groups:
        # Normalize RULER scores within group
        ruler_scores = [t.reward for t in g.trajectories]
        rmin, rmax = min(ruler_scores), max(ruler_scores)
        span = (rmax - rmin) if (rmax > rmin) else 1.0

        for t in g.trajectories:
            # Hard correctness gate
            gate = 1.0 if t.metrics["correct"] >= 0.5 else 0.0

            # Normalized RULER score
            ruler_norm = (t.reward - rmin) / span

            # Combined reward
            final_reward = gate * (alpha + beta * ruler_norm)
            t.reward = final_reward
    return groups
```

10. Key takeaways

- Reward functions define what behavior is reinforced
- Reward hacking arises from poorly specified absolute rewards
- Absolute scores are brittle for open ended agent tasks
- Relative ranking produces more stable and generalizable learning signals
- RULER turns subjective tasks into learnable feedback by comparing trajectories

Code example: Complete RULER training loop

```
# Full example: Countdown puzzle solving with RULER
for batch in training_iterator:
    # Generate rollouts
    groups = []
    for scenario in batch.items:
        group = art.TrajectoryGroup([
            rollout(model, scenario)
            for _ in range(2)
        ])
        groups.append(group)

    # Hard correctness check
    for group in groups:
        for traj in group.trajectories:
            reward, val, reason = judge_countdown_expression(
                scenario.target, scenario.nums,
```

```
                traj.final_answer.answer
            )
            traj.reward = float(reward)
            traj.metrics["correct"] = float(reward)

    # RULER relative ranking
    ruler_groups = []
    for group in groups:
        judged = await ruler_score_group(
            group, "openai/gpt-4.1"
        )
        ruler_groups.append(judged)

    # Combine hard gate + RULER
    hybrid = combine_rewards(ruler_groups, alpha=0.7, beta=0.3)

    # Train on combined rewards
    await model.train(hybrid, ...)
```

Section 3: Agent Trajectories and Rollout Design

Core idea: Rollout functions are the engine of learning. They turn agent interaction with an environment into trajectories that can be evaluated and optimized.

1. What ART introduces

ART formalizes how agents generate learning data. Instead of static prompt response pairs, ART operates on full interaction episodes.

Key shift:

- The agent does not just answer
- The agent acts within a defined scenario
- Interaction traces become training data

Code example: ART vs static prompt-response

```
# Static approach (supervised learning)
prompt = "What is 2+2?"
response = "4"  # Just the answer

# ART approach (reinforcement learning)
trajectory = art.Trajectory(
    messages_and_choices=[
        {"role": "system", "content": "You are a math agent."},
        {"role": "user", "content": "What is 2+2?"},
        {"role": "assistant", "content": "Let me calculate... 2+2=4"},
        # Tool calls, reasoning steps, context evolution
    ],
```

```
    reward=1.0,  # Evaluated based on process, not just answer
    metrics={"steps": 1, "tool_used": False}
)
# Full interaction trace becomes training data
```

2. What is a rollout

A rollout is one complete attempt by the agent to solve a task.

It includes:

- The initial scenario or task
- All system and user messages
- Every agent action or tool call
- The final outcome

When finished, the rollout produces a trajectory that can be scored.

Rollout structure and timeline

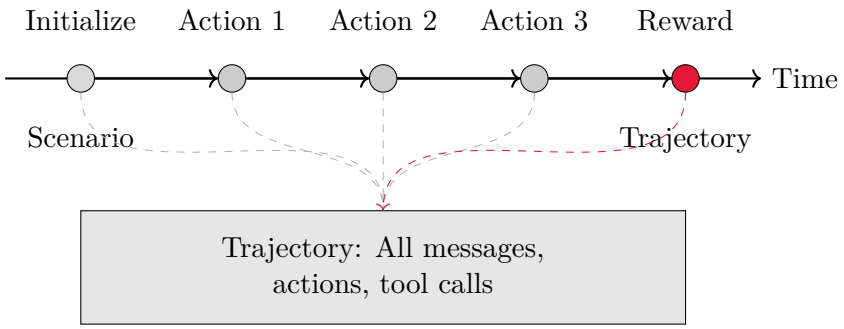

Figure 5: Rollout timeline showing initialization, sequential actions, reward assignment, and trajectory capture.

Code example: Complete rollout structure

```
# A rollout is one complete attempt to solve a task
async def rollout() -> art.Trajectory:
    # Initialize scenario
    trajectory = art.Trajectory(
        messages_and_choices=[
            {"role": "system", "content": "You are playing a game."},
            {"role": "user", "content": "What will your first move be?"},
        ],
        tools=[...],  # Available tools
        reward=0.0,   # Will be updated
        metrics={"num_rounds": 0}
    )

    # Agent acts (multiple turns)
```

```
    for round in range(10):
        response = await model.generate(trajectory.messages())
        trajectory.messages_and_choices.append(response)

        # Tool calls, environment updates, etc.
        # ... interaction continues ...

    # Final outcome determines reward
    trajectory.reward = compute_reward(trajectory)
    return trajectory  # Complete rollout
```

3. What is a trajectory

A trajectory is the full recorded history of a rollout.

It captures:

- Reasoning steps
- Tool usage
- Decisions made along the way
- Context evolution over time

Trajectories preserve how a result was achieved, not only what was produced.

Trajectories can also include **additional histories** for complex scenarios like multi-agent systems, history compaction, or preserving special tokens across turns.

Rollout vs Trajectory

Rollout	Trajectory
What happened	*How it happened*
(A single execution)	(The recorded path)
→ ACTING	→ REMEMBERING

Code example: Trajectory with tool calls and reasoning

```
# Trajectory captures full interaction history
trajectory = art.Trajectory(
    messages_and_choices=[
        {"role": "system", "content": "You are a game agent."},
        {"role": "user", "content": "What will your move be?"},

        # Assistant makes tool call
        {
            "finish_reason": "tool_calls",
            "index": 0,
```

```
            "message": {
                "role": "assistant",
                "tool_calls": [{
                    "id": "call_1",
                    "function": {"name": "play_move",
                                 "arguments": '{"move": "paper"}'}
                }]
            }
        },

        # Tool result
        {"role": "tool", "tool_call_id": "call_1",
         "content": "You win! Paper beats rock."},

        # Final response
        {
            "finish_reason": "stop",
            "index": 1,
            "message": {"role": "assistant",
                        "content": "Great! I chose paper."}
        }
    ],
    reward=0.9,
    metrics={"won": 1, "moves": 1},
    metadata={"result": "win", "strategy": "counter"}
)
# Captures reasoning, tool usage, and decision process
```

Code example: Trajectory with additional histories for sub-agents

```
# Trajectory with coordinator agent delegating to sub-agents
trajectory = art.Trajectory(
    messages_and_choices=[
        {"role": "user", "content": "Analyze codebase and fix bugs"},
        # Coordinator delegates to sub-agents via tool calls
        Choice(
            index=0,
            finish_reason="tool_calls",
            message={"role": "assistant",
                  "tool_calls": [{"id": "call_1",
                                  "function": {"name": "analyze_code"}}]}
        ),
        # ... tool results and final response ...
    ],
    additional_histories=[
        # Sub-agent 1: Code analyzer
        History(
            messages_and_choices=[
                {"role": "system", "content": "You are a code analysis expert."},
                {"role": "user", "content": "Find potential bugs in main.py"},
                Choice(index=0, finish_reason="stop",
                       message={"role": "assistant",
```

```
                                "content": "Found 3 potential issues..."})
            ]
        ),
        # Sub-agent 2: Bug fixer
        History(
            messages_and_choices=[
                {"role": "system", "content": "You are a bug fixing expert."},
                {"role": "user", "content": "Fix the null pointer issue"},
                Choice(index=0, finish_reason="stop",
                      message={"role": "assistant",
                               "content": "Fixed by adding null check..."})
            ]
        )
    ],
    reward=0.85,
    metrics={"sub_agents_used": 2, "bugs_fixed": 1}
)
# Additional histories preserve sub-agent interactions for training
```

4. The rollout function

The rollout function defines:

- The boundaries of the agent's world
- The scenario the agent is placed into
- When the task starts and ends
- How success or failure is measured

Before training code exists, the rollout function defines what learning is possible.

Code example: Rollout function defining the environment

```
@weave.op
async def rollout(model: art.Model,
                  scenario: Scenario) -> art.Trajectory:
    # 1. Define boundaries: game rules, tools, constraints
    tools = [{
        "type": "function",
        "function": {
            "name": "play_move",
            "parameters": {
                "properties": {
                    "move": {"enum": ["rock", "paper", "scissors"]}
                }
            }
        }
    }]

    # 2. Initialize scenario
    trajectory = art.Trajectory(
        messages_and_choices=[...],
```

```
        tools=tools,
        reward=0.0
    )

    # 3. Agent acts until task complete
    while not game_finished:
        response = await model.generate(...)
        trajectory.messages_and_choices.append(response)
        # ... environment updates ...

    # 4. Measure success/failure
    trajectory.reward = 1.0 if won else 0.0
    return trajectory
```

5. Designing a rollout function

Typical steps:

1. Initialize a concrete scenario instance
2. Provide the agent with the initial context
3. Let the agent act until the task is complete
4. Record all messages and actions
5. Assign a reward or attach evaluation metadata

Each execution produces one trajectory.

Code example: Step-by-step rollout design

```
async def rollout(model: art.Model, scenario: Scenario) -> art.Trajectory:
    # Step 1: Initialize concrete scenario instance
    game_state = initialize_game(scenario)

    # Step 2: Provide agent with initial context
    trajectory = art.Trajectory(
        messages_and_choices=[
            {"role": "system", "content": game_rules},
            {"role": "user", "content": scenario.question}
        ],
        tools=available_tools,
        reward=0.0
    )

    # Step 3: Let agent act until task complete
    while not is_complete(game_state):
        response = await model.generate(trajectory.messages())
        trajectory.messages_and_choices.append(response)

        # Execute tool calls, update state
        if has_tool_calls(response):
            result = execute_tool(response.tool_calls[0])
```

```
            trajectory.messages_and_choices.append(result)

    # Step 4: Record all messages and actions (already done above)
    # Step 5: Assign reward or evaluation metadata
    trajectory.reward = evaluate_outcome(game_state)
    trajectory.metrics["final_score"] = game_state.score
    trajectory.metadata["outcome"] = "win" if won else "lose"

    return trajectory  # One complete trajectory
```

Code example: Rollout with additional histories for sub-agents

```
async def rollout_with_sub_agents(model: art.Model,
                                  task: str) -> art.Trajectory:
    # Step 1: Initialize main coordinator scenario
    trajectory = art.Trajectory(
        messages_and_choices=[
            {"role": "system", "content": "You are a coordinator agent."},
            {"role": "user", "content": task}
        ],
        tools=[delegate_tool],
        reward=0.0
    )

    # Step 2: Coordinator acts and delegates
    response = await model.generate(trajectory.messages())
    trajectory.messages_and_choices.append(response)

    # Step 3: Execute sub-agent conversations
    sub_agent_histories = []
    for tool_call in extract_tool_calls(response):
        sub_task = parse_delegation(tool_call)
        # Run sub-agent rollout
        sub_history = await run_sub_agent(model, sub_task)
        sub_agent_histories.append(sub_history)

        # Add tool result to main trajectory
        trajectory.messages_and_choices.append({
            "role": "tool",
            "tool_call_id": tool_call["id"],
            "content": sub_history.get_final_result()
        })

    # Step 4: Attach sub-agent histories
    trajectory.additional_histories = sub_agent_histories

    # Step 5: Evaluate overall outcome
    trajectory.reward = evaluate_coordination(task, sub_agent_histories)
    trajectory.metrics["sub_agents_used"] = len(sub_agent_histories)

    return trajectory  # Trajectory with coordinator + sub-agents
```

6. Stateless and parallel execution

Rollout functions are designed to be:

- Stateless
- Thread safe
- Executable hundreds of times in parallel

This enables:

- Large scale trajectory generation
- Efficient group based evaluation
- Stable learning signals

Code example: Stateless parallel rollout generation

```python
# Rollout function is stateless - no shared state
async def rollout(model: art.Model, scenario: Scenario) -> art.Trajectory:
    # Each call is independent
    local_state = initialize_scenario(scenario)
    trajectory = art.Trajectory(...)
    # ... agent interaction ...
    return trajectory  # No side effects

# Can execute hundreds in parallel
trajectories = await art.gather_trajectories(
    (rollout(model, scenario) for _ in range(64)),
    max_exceptions=64
)

# Each rollout is independent and thread-safe
# No shared state means safe parallel execution
```

7. From interaction to learning signal

ART converts interaction into training data by:

- Running rollouts against a real or simulated environment
- Capturing interaction traces as trajectories
- Passing trajectories to an evaluation stage
- Using scores or rankings to update the policy

Interaction traces become the fundamental unit of learning.

Code example: Converting interaction to training signal

```
# Step 1: Run rollouts against environment
trajectories = await art.gather_trajectories(
    (rollout(model, scenario) for scenario in scenarios),
    max_exceptions=100
)

# Step 2: Capture interaction traces as trajectories
# (Already done in rollout function)
# Step 3: Pass to evaluation stage
groups = []
for scenario in scenarios:
    # Group trajectories by scenario
    group = art.TrajectoryGroup([
        t for t in trajectories
        if t.metadata["scenario_id"] == scenario.id
    ])
    groups.append(group)

# Step 4: Use scores/rankings to update policy
# RULER ranks trajectories within groups
judged_groups = []
for group in groups:
    judged = await ruler_score_group(group, "openai/gpt-4.1")
    judged_groups.append(judged)

# Train on ranked trajectories
await model.train(judged_groups, ...)
```

Code example: Training with trajectories containing additional histories

```
# Generate trajectories (some with additional histories)
trajectories = await art.gather_trajectories(
    (rollout(model, scenario) for scenario in scenarios),
    max_exceptions=100
)

# Trajectories may include additional histories for:
# - Sub-agent conversations
# - Preserved special tokens
# - History compaction
for traj in trajectories:
    if traj.additional_histories:
        print(f"Trajectory has {len(traj.additional_histories)} additional histories")
        # Each history tokenized separately during training

# Group and evaluate as normal
groups = [art.TrajectoryGroup([t for t in trajectories
                               if t.metadata["scenario_id"] == s.id])
          for s in scenarios]
```

```
# RULER evaluates full trajectory including additional histories
judged_groups = []
for group in groups:
    judged = await ruler_score_group(group, "openai/gpt-4.1")
    judged_groups.append(judged)

# Training processes all histories independently
# Training weights distributed across primary + additional histories
await model.train(judged_groups, config=art.TrainConfig(learning_rate=1e-5))
```

8. Reward and evaluation attachment

Once a rollout completes:

- Success or failure can be computed directly
- Or a judge model can evaluate the trajectory
- Or RULER can rank trajectories within a group

The evaluation result is attached to the trajectory and used for optimization.

Code example: Attaching evaluation to trajectory

```
# Option 1: Direct success/failure computation
trajectory.reward = 1.0 if game_won else 0.0
trajectory.metrics["correct"] = 1.0 if correct else 0.0

# Option 2: Judge model evaluation
judge_response = await judge_model.evaluate(trajectory)
trajectory.reward = judge_response.score
trajectory.metadata["judge_explanation"] = judge_response.reason

# Option 3: RULER relative ranking
group = art.TrajectoryGroup([traj1, traj2, traj3])
judged_group = await ruler_score_group(group, "openai/gpt-4.1")
# Each trajectory.reward now contains relative score

# Evaluation attached - ready for optimization
await model.train([judged_group], ...)
```

9. Why trajectories are superior to labeled data

- Labels only capture outcomes
- Trajectories capture decision processes
- Learning from trajectories enables strategy improvement
- Tool use and planning require process level feedback

This is why supervised fine tuning alone is insufficient for agents.

Code example: Trajectory vs label comparison

```
# Supervised learning: Only captures outcome
label = {
    "input": "What is 2+2?",
    "output": "4"  # Just the answer
}

# Trajectory: Captures decision process
trajectory = art.Trajectory(
    messages_and_choices=[
        {"role": "user", "content": "What is 2+2?"},
        {"role": "assistant", "content": "Let me think..."},
        # Tool call: calculator
        {"role": "tool", "content": "4"},
        {"role": "assistant", "content": "The answer is 4"}
    ],
    reward=1.0,
    metrics={"tool_used": True, "reasoning_steps": 2}
)

# Trajectory shows HOW the answer was reached
# Enables learning strategy, tool use, reasoning process
```

10. Additional histories for complex trajectories

Trajectories can contain multiple separate conversation histories within a single trajectory context. This enables training on:

- Non-linear conversation flows
- Agents that call sub-agents
- History compaction and summarization
- Preservation of special tokens across multi-turn conversations

Each trajectory has:

- A primary `messages_and_choices` sequence (the main conversation)
- An optional list of `additional_histories`, each with its own messages and optional tools

Code example: Preserving special tokens with additional histories

```
# Some models strip special tokens like <think> from earlier turns
# Additional histories preserve these tokens for training
trajectory = art.Trajectory(
    messages_and_choices=[
        {"role": "user", "content": "What is 2+2?"},
        Choice(
            index=0,
            finish_reason="stop",
```

```
            message={
                "role": "assistant",
                "content": "<think>I need to add 2 and 2</think>4"
            }
        )
    ],
    additional_histories=[
        History(
            messages_and_choices=[
                {"role": "user", "content": "What is 2+2?"},
                Choice(index=0, finish_reason="stop",
                      message={"role": "assistant", "content": "4"}),
                {"role": "user", "content": "What is 3+3?"},
                Choice(
                    index=0,
                    finish_reason="stop",
                    message={
                        "role": "assistant",
                        "content": "<think>I need to add 3 and 3</think>6"
                    }
                )
            ]
        )
    ],
    reward=0.9,
    metrics={"preserved_reasoning_tokens": True, "num_turns": 2}
)
# Each turn preserved separately maintains special tokens
```

Code example: Training coordinator agents with sub-agents

```
# Coordinator agent delegates to specialized sub-agents
trajectory = art.Trajectory(
    messages_and_choices=[
        {"role": "user", "content": "Research AI safety and write summary"},
        # Coordinator delegates via tool calls
        Choice(
            index=0,
            finish_reason="tool_calls",
            message={
                "role": "assistant",
                "tool_calls": [{
                    "id": "call_research",
                    "function": {"name": "delegate_to_sub_agent",
                                 "arguments": '{"sub_agent_type": "researcher"}'}
                }]
            }
        ),
        {"role": "tool", "tool_call_id": "call_research",
         "content": "Research completed: Found papers on alignment..."},
        # ... more coordinator actions ...
    ],
```

```
    tools=[{
        "type": "function",
        "function": {
            "name": "delegate_to_sub_agent",
            "parameters": {
                "properties": {
                    "sub_agent_type": {"type": "string"},
                    "task": {"type": "string"}
                }
            }
        }
    }],
    additional_histories=[
        # Researcher sub-agent conversation
        History(
            messages_and_choices=[
                {"role": "system", "content": "You are a research specialist."},
                {"role": "user", "content": "Research latest AI safety developments"},
                Choice(
                    index=0,
                    finish_reason="tool_calls",
                    message={
                        "role": "assistant",
                        "tool_calls": [{
                            "id": "call_search",
                            "function": {"name": "search",
                                         "arguments": '{"query": "AI safety 2024"}'}
                        }]
                    }
                ),
                {"role": "tool", "tool_call_id": "call_search",
                 "content": "Found papers on alignment, interpretability..."}
            ],
            tools=[{
                "type": "function",
                "function": {"name": "search", "parameters": {...}}
            }]
        ),
        # Writer sub-agent conversation
        History(
            messages_and_choices=[
                {"role": "system", "content": "You are a writing specialist."},
                {"role": "user", "content": "Write summary of AI safety research"},
                Choice(
                    index=0,
                    finish_reason="stop",
                    message={
                        "role": "assistant",
                        "content": "Summary: Recent AI safety developments..."
                    }
                )
            ]
        )
    ],
```

```
    reward=0.88,
    metrics={"sub_agents_used": 2, "tools_used": 3}
)
# Each sub-agent conversation stored as separate history
# Enables learning from both coordinator and sub-agent interactions
```

Code example: History compaction for long conversations

```
# Primary history contains compacted version
# Additional history preserves original detailed conversation
trajectory = art.Trajectory(
    messages_and_choices=[
        {"role": "system", "content": "You are a helpful assistant."},
        {
            "role": "user",
            "content": (
                "Compacted history: User asked about quantum entanglement, "
                "I explained basic concept of particle correlation."
            )
        },
        {"role": "user", "content": "Tell me more about the history"},
        Choice(
            index=0,
            finish_reason="stop",
            message={
                "role": "assistant",
                "content": "Quantum entanglement was first proposed by EPR in 1935..."
            }
        )
    ],
    additional_histories=[
        History(
            messages_and_choices=[
                {"role": "system", "content": "You are a helpful assistant."},
                {"role": "user", "content": "Explain quantum entanglement"},
                Choice(
                    index=0,
                    finish_reason="stop",
                    message={
                        "role": "assistant",
                        "content": "Quantum entanglement is a phenomenon where..."
                    }
                ),
                {"role": "user", "content": "Can you give me a simple example?"},
                Choice(
                    index=0,
                    finish_reason="stop",
                    message={
                        "role": "assistant",
                        "content": "Imagine two coins that are perfectly
                        ↪  anti-correlated..."
                    }
```

```
                )
            ]
        )
    ],
    reward=0.92,
    metrics={"history_compacted": True, "original_turns": 4, "compacted_turns": 3}
)
# Train on both compacted (efficient) and original (detailed) histories
```

When tokenized, each history is processed independently, and training weights are distributed across all histories in the trajectory.

Trajectory with additional histories

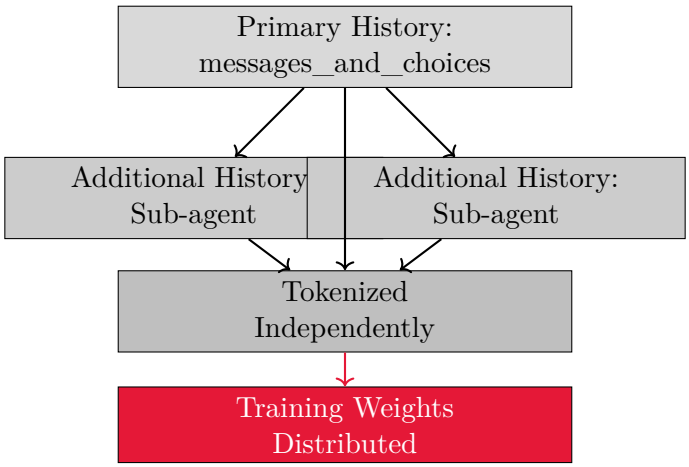

Figure 6: Trajectories can contain primary history plus additional histories (e.g., sub-agents), each tokenized independently for training.

11. Key takeaways

- Rollouts are complete episodes of agent behavior
- Trajectories record reasoning, actions, and tool use
- Rollout functions define what the agent can learn
- Stateless design enables scalable training
- ART turns interaction into optimization signal
- Additional histories enable complex multi-agent and multi-turn scenarios

Code example: Complete trajectory generation workflow

```
# 1. Generate multiple trajectories per scenario
async def generate_trajectories():
    groups = []
    for scenario in scenarios:
        # Multiple rollouts for same scenario
        group = art.TrajectoryGroup([
            await rollout(model, scenario)
            for _ in range(3)  # 3 rollouts per scenario
        ])
        groups.append(group)
    return groups

# 2. Evaluate with RULER (relative ranking)
groups = await generate_trajectories()
judged_groups = []
for group in groups:
    judged = await ruler_score_group(group, "openai/gpt-4.1")
    judged_groups.append(judged)

# 3. Train on ranked trajectories
await model.train(judged_groups,
                  config=art.TrainConfig(learning_rate=1e-5))

# Interaction → Trajectories → Ranking → Learning
```

Section 4: Optimization and Continuous Improvement

Core idea: Improving agents requires stability. Optimization must balance learning from new experience with preserving existing behavior.

1. Why stability matters in agent learning

Each policy update changes how the agent behaves. If updates are too aggressive:

- The agent forgets useful behaviors
- Training collapses or diverges
- Learning becomes noisy and unpredictable

Stable learning requires controlled policy updates.

2. KL divergence: the leash on learning

KL divergence measures how far a new policy deviates from the previous policy.

During optimization:

- New experience proposes an updated policy
- A KL penalty limits how far the update can move

- Learning balances improvement with stability

KL acts as a counterforce against destructive updates.

3. The noise problem with PPO

PPO relies on a critic model to estimate value. For complex agent tasks:

- The critic is often wrong
- Advantage estimates become noisy
- Policy updates move in the wrong direction

This introduces instability and slows learning.

4. Group based optimization: the key shift

Group based methods avoid guessing. Instead of learning from isolated samples:

- Multiple trajectories are generated per prompt
- Performance is evaluated relative to the group
- The group average becomes the baseline

Learning becomes comparative rather than absolute.

Group-based optimization

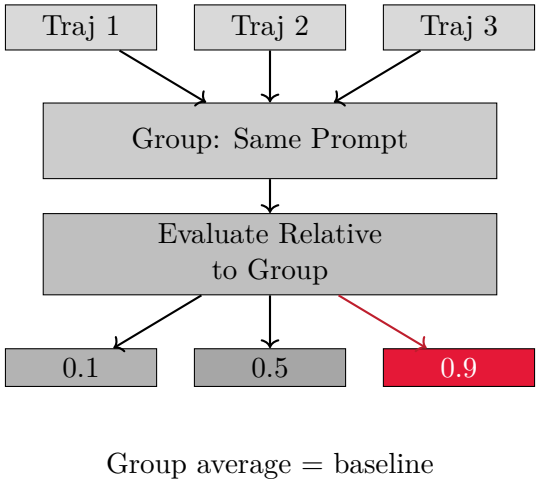

Figure 7: Group-based optimization: multiple trajectories per prompt are evaluated relative to each other, with group average as baseline.

5. Group Relative Policy Optimization (GRPO)

GRPO works by:

- Generating a group of trajectories
- Scoring them using a judge or reward function

- Normalizing scores within the group
- Updating the policy using token level importance ratios

Key property: The baseline is real data, not a learned critic.

6. Group Sequence Policy Optimization (GSPO)

GSPO extends GRPO to the sequence level.

Differences from GRPO:

- Optimization operates on full sequences
- Importance ratios are applied per trajectory
- Reward granularity aligns with how tasks are judged

This improves stability for long and complex trajectories.

GRPO vs GSPO

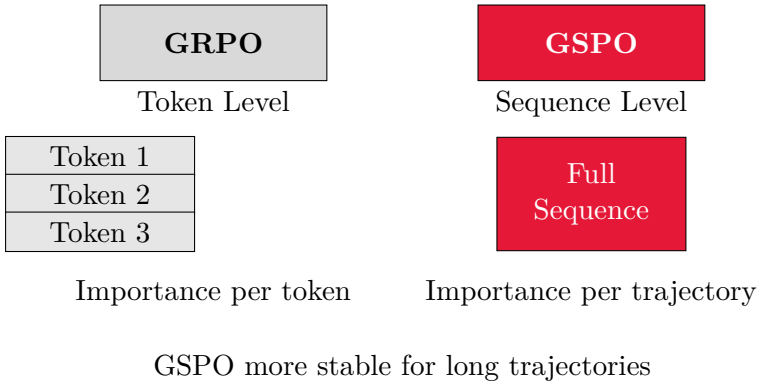

Figure 8: GRPO operates at token level (importance per token), while GSPO operates at sequence level (importance per trajectory).

7. GRPO vs GSPO

Aspect	Difference
Learning level	GRPO operates at token level, GSPO at sequence level
Baseline	Both use group normalized advantages
Stability	GSPO is more stable for long trajectories
Use case	GSPO preferred for tool heavy and MoE models

8. Mixture of Experts (MoE) volatility

MoE models route tokens through different experts. After a policy update:

- Activated experts can change significantly
- Small updates cause large behavior shifts

- Training becomes unstable

This expert volatility is amplified with depth.

9. GSPO and MoE stability

GSPO mitigates MoE instability because:

- Learning happens at the sequence level
- Expert routing noise is averaged out
- No routing replay is required

This makes GSPO infrastructure friendly and robust.

10. Checkpoint forking

Training progresses through checkpoints. If training collapses:

- An earlier stable checkpoint can be reused
- New branches can explore different parameters
- Experiments don't need to restart from scratch

Checkpoint forking creates a branching training timeline.

11. Best practices for iterative improvement

- Monitor relative scores, not raw rewards
- Validate on held out scenarios
- Increase scenario diversity gradually
- Prefer GSPO for long horizon tasks
- Treat checkpoints as first class state

12. Key takeaways

- Stable optimization is critical for agent learning
- KL divergence constrains destructive updates
- PPO suffers from noisy value estimation
- Group based optimization replaces critics with data
- GSPO stabilizes learning for complex agents
- Continuous improvement relies on checkpoint forking

Section 5: Teaching Agents to Master Tools Automatically

Core idea: Tool use should be learned, not scripted. Agents can be trained to discover, select, and use tools automatically through reinforcement learning.

1. Why tool use is hard for agents

Tool use is an open ended decision problem. The agent must decide:

- Whether to use a tool at all
- Which tool to select
- How to parameterize the tool call
- How to recover from tool errors

Binary correctness metrics fail to capture tool quality.

2. Model Context Protocol (MCP)

Model Context Protocol acts as a standardized interface between agents and tools.

MCP provides:

- Tool discovery
- Tool schemas
- A unified interaction protocol

Agents can ask an MCP server what tools exist and how to use them.

Code example: Connecting to MCP server and discovering tools

```
# Connect to MCP server
async def list_tools_and_resources():
    async with streamablehttp_client(MCP_URL, headers=HEADERS) as (read, write, _):
        async with ClientSession(read, write) as session:
            await session.initialize()
            # Discover available tools
            tools = await session.list_tools()
            resources = await session.list_resources()
            return tools, resources

# Get tool schemas for agent
tools_result, resources_result = await list_tools_and_resources()
tool_schemas = [
    {
        "type": "function",
        "function": {
            "name": tool.name,
            "description": tool.description,
            "parameters": tool.inputSchema
        }
    }
    for tool in tools_result.tools
]
# Agent now knows what tools are available
```

3. MCP as the USB-C port for agents

MCP abstracts away individual APIs. Instead of writing custom integrations:

- The agent speaks one protocol
- Any MCP compatible tool becomes usable
- Tool ecosystems become composable

This enables scalable tool ecosystems.

Code example: Calling MCP tools through standardized interface

```
async def call_mcp_tool(tool_name: str, arguments: dict):
    async with mcp_session() as session:
        return await session.call_tool(tool_name, arguments)

# Same interface works for any MCP-compatible tool
result = await call_mcp_tool(
    "web_search_exa",
    {"query": "Qwen2.5 context length", "numResults": 3}
)
# No need to know Exa API specifics - MCP abstracts it
```

4. ART and MCP: the power couple

ART provides:

- Reinforcement learning infrastructure
- Rollout generation
- Optimization loops

MCP provides:

- Real tool environments
- Ground truth success or failure
- Tool schemas and constraints

Together they enable learning tool use from interaction.

Code example: ART rollout with MCP tools

```
# MCP provides tool schemas
tools_result, _ = await list_tools_and_resources()
tool_schemas = [convert_to_openai_format(t) for t in tools_result.tools]

# ART rollout uses MCP tools
@weave.op()
async def rollout(model: art.Model, scenario: McpScenario) -> art.Trajectory:
    traj = art.Trajectory(
```

```
        messages_and_choices=[
            {"role": "system", "content": "You are an MCP agent."},
            {"role": "user", "content": scenario.task_description}
        ],
        tools=tool_schemas,  # MCP tools available to agent
        reward=0.0
    )

    # Agent interacts with MCP tools
    response = await model.generate(traj.messages())
    if response.tool_calls:
        for tool_call in response.tool_calls:
            # Call MCP tool
            result = await call_mcp_tool(tool_call.name, tool_call.arguments)
            traj.messages_and_choices.append({
                "role": "tool",
                "tool_call_id": tool_call.id,
                "content": result
            })

    return traj
# ART handles RL, MCP handles tool execution
```

5. Scenario generation for tool learning

Instead of hand writing prompts:

- The system queries the MCP server for available tools
- ART generates a curriculum of tool use scenarios
- Scenarios cover simple, complex, and edge cases

Scenario generation replaces manual prompt engineering.

Code example: Generating scenarios from MCP tools

```
# Query MCP server for available tools
tools_result, resources_result = await list_tools_and_resources()
tools_list = [
    {"name": t.name, "description": t.description,
     "parameters": t.inputSchema}
    for t in tools_result.tools
]

# ART generates scenarios automatically
scenario_collection = await generate_scenarios(
    tools=tools_list,
    resources=resources_list,
    num_scenarios=32,  # Generate training scenarios
    generator_model="openai/o4-mini",
    generator_api_key=OPENROUTER_API_KEY
)
```

```
scenarios = [
    {"task": s.task, "difficulty": s.difficulty}
    for s in scenario_collection.scenarios
]
# Scenarios cover tool use cases without manual writing
```

6. Types of training scenarios

Generated scenarios include:

- Single tool calls
- Multi step tool workflows
- Invalid inputs and error cases
- Creative tool combinations

This ensures coverage of real production behavior.

Code example: Types of generated scenarios

```
# Generated scenarios include various complexity levels:
scenarios = [
    # Simple: Single tool call
    {"task": "Search for information about Python async", "difficulty": 1},

    # Medium: Multi-step workflow
    {"task": "Compare two products and create a comparison table", "difficulty": 3},

    # Complex: Multiple tools, error handling
    {"task": "Research code examples, analyze them, and write a guide", "difficulty": 4},

    # Edge cases: Invalid inputs, rate limits
    {"task": "Search with invalid parameters and recover", "difficulty": 2}
]

# Scenarios automatically cover:
# - Single tool calls
# - Multi-step workflows
# - Error cases
# - Creative tool combinations
```

7. RULER for tool evaluation

Tool use can't be evaluated with simple success flags.

RULER evaluates trajectories based on:

- Task accomplishment
- Tool selection quality

- Efficiency of tool usage
- Error handling and recovery

No human labels are required.

Code example: RULER evaluation for tool use

```
# Generate trajectories with tool use
groups = await art.gather_trajectory_groups(
    (art.TrajectoryGroup(
        rollout(model, scenario)
        for _ in range(2)  # Multiple rollouts per scenario
    ) for scenario in scenarios)
)

# RULER evaluates tool use quality automatically
scored_groups = []
for group in groups:
    judged_group = await ruler_score_group(
        group,
        judge_model="openai/o4-mini",
        debug=True
    )
    # RULER considers:
    # - Task accomplishment
    # - Tool selection quality
    # - Efficiency of tool usage
    # - Error handling
    scored_groups.append(judged_group)

# No human labels needed - RULER judges from outputs
```

8. The tool learning loop

The learning loop:

1. Generate tool use scenarios
2. Run agent rollouts against MCP tools
3. Capture trajectories
4. Rank trajectories using RULER
5. Update the policy with group based optimization

Tool mastery emerges from repeated interaction.

Tool learning loop

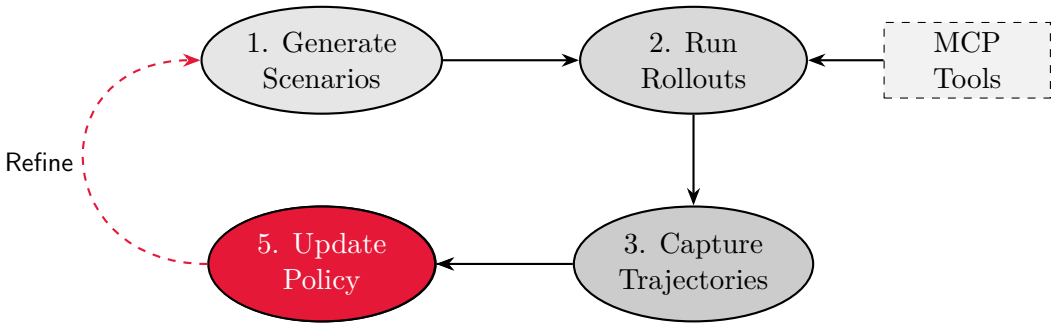

Figure 9: The tool learning loop: generate scenarios, run rollouts with MCP tools, capture trajectories, rank with RULER, and update policy.

Code example: Complete tool learning loop

```
# Step 1: Generate tool use scenarios
tools_result, _ = await list_tools_and_resources()
scenarios = await generate_scenarios(
    tools=tools_list,
    num_scenarios=32
)

# Step 2: Run agent rollouts against MCP tools
@weave.op()
async def rollout(model: art.Model, scenario: McpScenario) -> art.Trajectory:
    traj = art.Trajectory(
        messages_and_choices=[
            {"role": "system", "content": "You are an MCP agent."},
            {"role": "user", "content": scenario.task_description}
        ],
        tools=mcp_tool_schemas
    )

    # Agent uses MCP tools
    for turn in range(MAX_TURNS):
        response = await model.generate(traj.messages())
        if response.tool_calls:
            for tool_call in response.tool_calls:
                result = await call_mcp_tool(tool_call.name, tool_call.args)
                traj.messages_and_choices.append({
                    "role": "tool",
                    "tool_call_id": tool_call.id,
                    "content": result
                })
        if task_completed:
            break

    return traj

# Step 3: Capture trajectories
```

```
groups = await art.gather_trajectory_groups(
    (art.TrajectoryGroup(
        rollout(model, scenario) for _ in range(2)
    ) for scenario in scenarios)
)

# Step 4: Rank trajectories using RULER
scored_groups = []
for group in groups:
    judged = await ruler_score_group(group, "openai/o4-mini")
    scored_groups.append(judged)

# Step 5: Update policy with group-based optimization
await model.train(
    scored_groups,
    config=art.TrainConfig(learning_rate=1e-5)
)
# Tool mastery emerges from repeated interaction
```

9. Production as data

Every production tool call produces:

- A real world trajectory
- Ground truth success or failure
- New training data

Production usage becomes the most valuable dataset.

Code example: Testing trained model on new tasks

```
# Test model on validation scenarios
val_scenarios = [
    McpScenario(task_description=task, max_turns=MAX_TURNS)
    for task in validation_tasks
]

# Each test produces a real-world trajectory
for scenario in val_scenarios:
    result_trajectory = await rollout(model, scenario)

    # Trajectory contains:
    # - Real tool calls
    # - Actual tool results
    # - Ground truth success/failure
    # - New training data

    print(f"Task: {scenario.task_description}")
    print(f"Success: {result_trajectory.metrics['success']}")
    print(f"Turns: {result_trajectory.metrics['num_turns']}")

    # Production usage = most valuable dataset
```

```
    # Can be added back to training set
```

10. The self improving flywheel

- Agent uses tools in production
- Interactions are logged as trajectories
- RULER evaluates behavior offline
- The optimizer updates the policy
- The agent improves continuously

No human feedback loop is required.

Self-improving flywheel

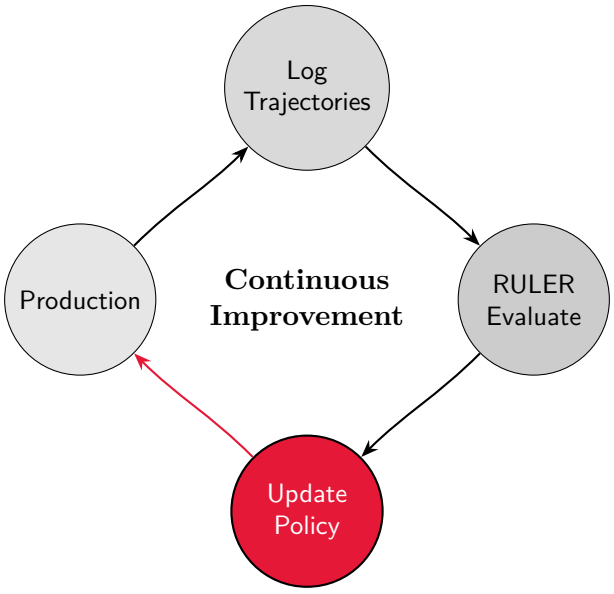

Figure 10: Self-improving flywheel: production usage generates trajectories, RULER evaluates offline, policy updates, and agent improves continuously.

Code example: Self-improving flywheel with MCP

```python
# 1. Agent uses tools in production
production_trajectories = []
for user_request in production_requests:
    traj = await rollout(model, McpScenario(task_description=user_request))
    production_trajectories.append(traj)

# 2. Interactions logged as trajectories
```

```
# (Already done in step 1)

# 3. RULER evaluates behavior offline
production_groups = [
    art.TrajectoryGroup([traj1, traj2])
    for traj1, traj2 in zip(production_trajectories[::2],
                            production_trajectories[1::2])
]

scored_groups = []
for group in production_groups:
    judged = await ruler_score_group(group, "openai/o4-mini")
    scored_groups.append(judged)

# 4. Optimizer updates policy
await model.train(scored_groups, config=art.TrainConfig(learning_rate=1e-5))

# 5. Agent improves continuously
# Loop repeats: production → evaluation → training → better agent
# No human feedback required
```

11. Best practices

- Ensure scenario diversity
- Include error cases explicitly
- Monitor tool efficiency metrics
- Validate on held out tools
- Treat production data as training data

Code example: Reward shaping and deliverable gates

```
# Reward shaping: penalize inefficiency and encourage completion
def compute_shaped_reward(traj, base_reward: float) -> float:
    num_turns = traj.metrics.get("num_turns", 0)
    task_completed = traj.metrics.get("task_completed", False)
    ran_out = traj.metrics.get("ran_out_of_turns", False)

    r = base_reward

    # Penalize excessive turns (encourage efficiency)
    r -= 0.015 * num_turns

    # Strong penalty for not completing task
    if not task_completed:
        r -= 0.35

    # Penalize running out of turns
    if ran_out:
        r -= 0.25
```

```
    # Bonus for early completion
    if task_completed and num_turns <= 3:
        r += 0.10

    return max(min(r, 1.0), -1.0)

# Deliverable gate: ensure agent produces output before completion
def has_deliverable(traj, task_desc: str) -> bool:
    # Check for substantial content (1200+ chars)
    # Check for structured output (lists, tables, code)
    # Check for no future tense ("will do" vs "did")
    assistant_text = extract_assistant_text(traj)

    if len(assistant_text) < 1200:
        return False

    if not has_structure(assistant_text):  # lists, tables, headings
        return False

    if has_future_tense(assistant_text):
        return False

    return True

# Apply gates in rollout
if tool_name == "complete_task":
    if not has_deliverable(traj, scenario.task_description):
        # Refuse premature completion
        traj.messages_and_choices.append({
            "role": "tool",
            "tool_call_id": tool_call.id,
            "content": "REFUSED: Produce full deliverable first"
        })
        continue
```

12. Key takeaways

- Tool use is a learning problem, not a prompt problem
- MCP standardizes agent tool access
- ART trains agents from real interaction
- RULER evaluates tool quality via comparison
- Production usage drives continuous improvement